

RECIPE SHELTER

Site Web de Recettes de Cuisine Collaboratif

DOCUMENTATION TECHNIQUE — BLOC 2 : BACK-END

Architecture · Base de données · Sécurité · API · Tests

Élément	Détail
Runtime	Node.js LTS
Framework HTTP	Express 5
Langage	TypeScript 5.9 (strict mode, ESM)
Base de données	MySQL 8 — pool mysql2/promise
Authentification	JWT signé (jsonwebtoken) · cookie HttpOnly
Hachage	bcrypt (coût 12)
Emails	Nodemailer (SMTP)
Tests	Node.js test runner natif — *.test.ts
Qualité	ESLint + Husky + lint-staged

1. Architecture en couches

1.1 Vue d'ensemble

Le back-end adopte une architecture en trois couches clairement séparées, inspirée des patterns DDD (Domain-Driven Design) et des bonnes pratiques Node.js. Chaque couche a une responsabilité unique et ne communique qu'avec la couche adjacente.

Couche	Responsabilité
Controllers (API)	Réception des requêtes HTTP, parsing et validation des corps (DTOs), appel des services, construction des réponses. Aucune logique métier.
Services (Métier)	Logique métier, règles de gestion, orchestration des repositories. C'est ici que sont appliquées les règles : qui peut voir quoi, quand une recette peut être soumise, etc.
Repositories (Données)	Accès à la base de données MySQL. Chaque repository expose une interface TypeScript, implémentée séparément (pattern Repository). Retourne des types métier, jamais de RowDataPacket brut.

1.2 Principes POO appliqués

- Encapsulation : chaque service expose uniquement les méthodes publiques nécessaires. Les méthodes de contrôle d'accès (isOwner, canEditRecipe...) sont privées.
- Injection de dépendances : tous les services et repositories reçoivent leurs dépendances via le constructeur. Aucune dépendance n'est instanciée dans les classes.
- Interfaces séparées des implémentations : chaque repository est défini par une interface TypeScript (UserRepository, RecipeRepository...) et implémenté par une classe MySQL concrète. Cela permet de substituer facilement l'implémentation (tests, changement de BDD).
- Composition au démarrage : toutes les dépendances sont instanciées et injectées dans app.ts — un seul point d'entrée pour l'assemblage de l'application.

1.3 Structure des dossiers

Dossier	Contenu
src/api/	Controllers, routes Express et DTOs par domaine (auth, recipes, comments, favorites, admin...)
src/services/	Classes de service métier par domaine
src/repositories/	Interfaces et implémentations MySQL par entité
src/middlewares/	Middlewares Express : requireAuth, requireAdmin, rateLimiter, errorHandler, notFound
src/db/	Pool de connexions MySQL, helpers de requête et de transaction
src/utils/	Helpers partagés : errors, env, logger, pagination, session-cookie
database/	Scripts SQL : migration (1_create_schema.sql), seed minimal, seed de démo
tests/	Tests unitaires (*.test.ts) et fichiers HTTP de test

2. Base de données MySQL

2.1 Schéma

La base de données `recipe_shelter` comprend 17 tables organisées en trois groupes : les entités utilisateur (Users, Roles, logs de modération, tokens d'email et de réinitialisation), les entités recette (Recipes, RecipeCategories, RecipeIngredients, RecipeSteps, RecipeEquipments, RecipeTags) et les données de référence (Ingredients, Equipments, Tags, Categories).

Toutes les tables utilisent le moteur InnoDB, la collation `utf8mb4_unicode_ci` pour le support Unicode complet (emojis, caractères accentués) et des clés étrangères avec contraintes CASCADE ou RESTRICT selon les cas.

2.2 Tables principales

Table	Colonnes clés	Particularités
Users	Id, Mail, Username, Password, RoleId, Status, EmailValidatedAt, BannedAt	Status : ENUM inactive/active/banned. Index sur Mail (unique), Username (unique), Status.
Recipes	Id, UserId, CategoryId, Title, Slug, Status, PrepTimeMinutes, CookTimeMinutes, PublishedAt	Status : ENUM draft/pending/published/rejected/archived. Index sur Status, PublishedAt, UserId.
RecipeIngredients	RecipeId, IngredientId, Quantity, Unit, Note, SortOrder	Clé primaire composite (RecipeId, IngredientId). SortOrder pour l'ordre d'affichage.
RecipeSteps	RecipeId, StepNumber, Description	Clé primaire composite (RecipeId, StepNumber).
Comments	Id, RecipeId, UserId, ParentCommentId, Rating, Comment, ModeratedAt, DeletedAt	ParentCommentId : 1 niveau de réponse uniquement. Rating uniquement sur les commentaires racine.
Favorites	UserId, RecipeId, CreatedAt	Clé primaire composite (UserId, RecipeId). Relation many-to-many Users ↔ Recipes.
EmailValidations	Id, UserId, TokenHash, ExpiresAt, UsedAt	TokenHash : CHAR(64) SHA-256. Index unique sur TokenHash.
PasswordResets	Id, UserId, TokenHash, ExpiresAt, UsedAt	Même structure qu'EmailValidations.
UserModerationLogs	Id, UserId, AdminId, Action, Reason, CreatedAt	Traçabilité des bans/unbans. Action : ENUM ban/unban.

2.3 Optimisations SQL

- Index sur les colonnes fréquemment filtrées : Status et PublishedAt sur Recipes (requêtes de listing), Status sur Users (vérification de compte actif), RoleId sur Users.
- Requêtes paramétrées systématiques via mysql2 — aucune concaténation de chaînes dans les requêtes SQL, protection contre les injections SQL.
- Pool de connexions mysql2 avec waitForConnections et keepAlive — réutilisation des connexions pour éviter le coût d'établissement à chaque requête.
- Requête de recherche avancée avec construction dynamique du WHERE : chaque filtre actif (q, categoryId, tagIds, ingredientIds, maxTotalTime) ajoute une clause paramétrisée, évitant les requêtes inutilement complexes quand les filtres sont vides.
- Pagination avec COUNT(*) séparé : une requête compte le total, une autre récupère la page. Évite le SQL_CALC_FOUND_ROWS qui peut être lent sur de grandes tables.

2.4 Scripts de base de données

Script	Rôle
database/migrations/1_create_schema.sql	Crée toutes les tables, clés étrangères et index. Point de départ de tout environnement.
database/seed.sql	Insère les données de référence minimales : rôles, catégories, ingrédients courants, tags, ustensiles.
database/seed_demo.sql	Insère des recettes, commentaires et favoris de démonstration pour tester l'application.
database/reset.sql	Supprime et recrée la base complète + seed minimal. Commande : npm run db:reset.
database/reset_demo.sql	Supprime et recrée la base complète + seed de démo. Commande : npm run db:reset:demo.

3. Sécurité

3.1 Authentification JWT

L'authentification repose sur des JSON Web Tokens (JWT) signés avec un secret de 256 bits minimum (HMAC-SHA256). Le token est stocké dans un cookie HttpOnly nommé `rs_session`, ce qui le rend inaccessible au JavaScript côté client et protège contre les attaques XSS.

Paramètre	Valeur et justification
Algorithme	HMAC-SHA256 (jsonwebtoken par défaut)
Payload	{ sub: userId, username, roleId, status } — le minimum nécessaire
Expiration	7 jours par défaut (configurable via <code>JWT_EXPIRES_IN</code>)
Stockage	Cookie HttpOnly, SameSite=lax, Secure=true en production
Cookie name	<code>rs_session</code> (configurable via <code>AUTH_SESSION_COOKIE_NAME</code>)
SameSite	lax — protège contre le CSRF pour les requêtes cross-site non intentionnelles

Le middleware `requireAuth` vérifie le cookie à chaque requête protégée : extraction du token, vérification de la signature, décodage du payload, puis vérification en base que le compte existe et est actif. Ce dernier point garantit que les comptes bannis ou supprimés n'ont plus accès même si leur token est encore valide.

3.2 Hachage des mots de passe

Les mots de passe sont hachés avec `bcrypt` avant stockage. Le coût est configuré à 12 par défaut (environ 300ms de calcul), ce qui offre un bon équilibre entre sécurité et performance. Les mots de passe ne sont jamais stockés, transmis ou journalisés en clair.

- Validation de la politique de mot de passe avant hachage : minimum 8 caractères, maximum 128 caractères.
- Comparaison via `bcrypt.compare` — résistante aux attaques par timing grâce à la comparaison en temps constant.

3.3 Validation d'email et réinitialisation de mot de passe

Les tokens de validation d'email et de réinitialisation de mot de passe sont générés avec `crypto.randomBytes(32)`, puis hachés en SHA-256 avant stockage (`TokenHash CHAR(64)`). Le token brut est envoyé uniquement par email, jamais stocké. En cas de compromission de la base, les tokens sont inutilisables.

- Expiration : chaque token a une date d'expiration (`ExpiresAt`). Les tokens expirés ou déjà utilisés (`UsedAt` non null) sont rejetés.
- Usage unique : le token est marqué `UsedAt = NOW()` à la première utilisation.

3.4 Rate limiting

Un middleware de rate limiting maison (sans dépendance externe) est appliqué sur les routes sensibles d'authentification. Il suit les tentatives par IP + méthode + route et renvoie HTTP 429 avec les headers RateLimit-Limit, RateLimit-Remaining et Retry-After conformes à la RFC 6585.

3.5 CORS

La configuration CORS n'accepte pas le wildcard (*) lorsque credentials: true est activé — Express lève une erreur au démarrage si CORS_ALLOWED_ORIGINS contient *. Seules les origines listées explicitement dans la variable d'environnement sont autorisées.

3.6 Gestion des erreurs HTTP

Toutes les erreurs métier sont construites via des factories typées (badRequest, unauthorized, forbidden, notFound, conflict) qui produisent des HttpError avec un statusCode et un code métier lisible (AUTH_INVALID_CREDENTIALS, RECIPES_NOT_FOUND...). Le errorHandler Express centralise la réponse JSON et journalise les erreurs 5xx.

4. API REST

4.1 Convention

Toutes les routes sont préfixées par `/api/v1`. Les réponses sont en JSON. Les erreurs suivent le format `{ error: { message, code } }`. La pagination suit le format `{ data: [...], total, page, limit, totalPages }`.

4.2 Routes par domaine

Authentification — `/api/v1/auth`

Méthode + Route	Accès	Description
POST <code>/register</code>	Public	Inscription — crée un compte inactif et envoie l'email de validation
POST <code>/login</code>	Public	Connexion — vérifie les identifiants, pose le cookie <code>rs_session</code>
POST <code>/logout</code>	Connecté	Déconnexion — supprime le cookie
GET <code>/me</code>	Connecté	Retourne l'AuthContext du token courant
POST <code>/forgot-password</code>	Public	Demande de réinitialisation — envoie un email avec token
POST <code>/reset-password</code>	Public	Réinitialise le mot de passe via token
POST <code>/validate-email</code>	Public	Valide l'email via token — passe le compte en active
POST <code>/resend-validation-email</code>	Public	Renvoie l'email de validation

Recettes — `/api/v1/recipes`

Méthode + Route	Accès	Description
GET <code>/recipes</code>	Public	Liste paginée des recettes publiées
GET <code>/recipes/search</code>	Public	Recherche avancée : <code>q</code> , <code>categoryId</code> , <code>tagIds</code> , <code>ingredientIds</code> , <code>maxTotalTimeMinutes</code>
GET <code>/recipes/recent</code>	Public	Dernières recettes publiées (limit)
GET <code>/recipes/:slug</code>	Public	Détail d'une recette publiée par slug
GET <code>/me/recipes</code>	Connecté	Mes recettes (tous statuts) avec pagination
GET <code>/me/recipes/:id</code>	Connecté	Détail d'une de mes recettes
POST <code>/me/recipes</code>	Connecté	Créer une recette (brouillon)
PUT <code>/me/recipes/:id</code>	Connecté	Modifier un brouillon
POST <code>/me/recipes/:id/submit</code>	Connecté	Soumettre à la modération
POST <code>/me/recipes/:id/archive</code>	Connecté	Archiver une recette publiée ou rejetée

Administration — /api/v1/admin

Méthode + Route	Accès	Description
GET /admin/recipes	Admin	Liste des recettes en attente de modération
GET /admin/recipes/:id	Admin	Détail d'une recette à modérer
POST /admin/recipes/:id/approve	Admin	Approuver et publier une recette
POST /admin/recipes/:id/reject	Admin	Rejeter avec motif
GET /admin/comments/moderated	Admin	Commentaires modérés
PATCH /admin/comments/:id/moderate	Admin	Masquer un commentaire
GET /admin/users/banned	Admin	Liste des utilisateurs bannis
POST /admin/users/:id/ban	Admin	Bannir un utilisateur avec motif
POST /admin/users/:id/unban	Admin	Débannir un utilisateur

5. Patterns et choix techniques

5.1 Pattern DTO (Data Transfer Object)

Chaque controller dispose de fonctions `parseXxxBody()` qui valident et transforment le corps de la requête avant de l'envoyer au service. Ces fonctions lèvent une `HttpError 400` si les données sont invalides. Ce pattern garantit que les services ne reçoivent jamais de données non validées.

```
// Exemple : parseLoginBody
export function parseLoginBody(body: unknown): { mail: string; password:
string } {
  const mail = getString(body, 'mail');
  const password = getString(body, 'password');
  if (!mail || !password) throw badRequest('Missing fields',
'AUTH_MISSING_FIELDS');
  return { mail, password };
}
```

5.2 Pattern Mapper

Chaque repository dispose d'un mapper qui transforme les `RowDataPacket MySQL` (noms de colonnes en `PascalCase`) en types `TypeScript` du domaine (noms en `camelCase`). Ce découplage garantit que les types métier ne dépendent pas de la structure de la base de données.

```
// Exemple : mapUser(row: UserRow): User
export function mapUser(row: UserRow): User {
  return { id: row.Id, mail: row.Mail, username: row.Username, ... };
}
```

5.3 asyncHandler

Tous les handlers Express sont wrappés dans `asyncHandler()`, qui capture les exceptions des fonctions `async` et les passe à `next()`. Cela évite les `try/catch` répétés dans chaque controller et garantit que toutes les erreurs remontent au `errorHandler` centralisé.

5.4 Pagination

Un utilitaire de pagination standardise le calcul de l'offset et la construction du résultat paginé pour toutes les listes. Il accepte les paramètres `page` et `limit` (avec valeurs par défaut et limites max), et retourne `{ data, total, page, limit, totalPages }`.

5.5 SmtplibService

Le service d'email est une classe `SmtplibService` qui encapsule `Nodemailer`. Elle est injectée dans les services qui en ont besoin (`EmailValidationService`, `PasswordResetService`, `ContactService`). La configuration SMTP est entièrement externalisée dans les variables d'environnement.

6. Tests unitaires

6.1 Stratégie

Les tests unitaires couvrent les DTOs (parsing et validation des corps de requête) et les controllers d'authentification. Ils utilisent le test runner natif de Node.js (node --test), sans dépendance externe de test. Chaque test vérifie le comportement avec des entrées valides et invalides.

6.2 Périmètre

Fichier testé	Type	Ce qui est vérifié
auth.dto.test.ts	DTO	parseLoginBody, parseRegisterBody : champs manquants, formats invalides, normalisation
auth.controller.test.ts	Controller	Login avec credentials valides/invalides, gestion des erreurs 401, 400
recipes.dto.test.ts	DTO	parseCreateRecipeBody : titre requis, temps de préparation, ingrédients
comments.dto.test.ts	DTO	parseCreateCommentBody : commentaire vide, rating hors plage
users.dto.test.ts	DTO	parseUpdateProfileBody : username trop court, email invalide
admin.recipes.dto.test.ts	DTO	parseApproveBody, parseRejectBody : motif de rejet requis
admin.comments.dto.test.ts	DTO	parseModerationBody
admin.users.dto.test.ts	DTO	parseBanBody : raison requise
contact.dto.test.ts	DTO	parseContactBody : email, message, sujet

6.3 Commandes

Commande	Description
npm run test	Exécute tous les tests avec node --test et tsx pour TypeScript
npm run lint	Vérifie le code avec ESLint
npm run lint:fix	Corrige automatiquement les problèmes ESLint réparables
npm run build	Compile TypeScript vers dist/

7. Bilan des choix techniques

Choix	Justification
Node.js + Express 5	Runtime JavaScript côté serveur. Express 5 ajoute le support natif des <code>async/await</code> dans les handlers (plus besoin de <code>try/catch</code> ou de wrapper).
TypeScript strict mode	Sécurité des types sur l'ensemble du back-end. Détection des erreurs à la compilation. Cohérence avec le front-end Angular également en TypeScript.
Architecture en 3 couches	Séparation claire des responsabilités. Chaque couche est testable indépendamment. Les services ne connaissent pas Express, les repositories ne connaissent pas la logique métier.
Interfaces Repository	Découplage de l'implémentation MySQL. Facilite les tests (injection d'une implémentation mock) et un éventuel changement de BDD.
JWT en cookie <code>HttpOnly</code>	Plus sécurisé que le <code>localStorage</code> (inaccessible au JS côté client). <code>SameSite=lax</code> protège contre le CSRF sur le même domaine.
Rate limiter maison	Aucune dépendance externe. Implémentation simple basée sur une Map en mémoire avec TTL. Adapté à une instance unique de serveur.
<code>mysql2/promise</code> + pool	Client MySQL moderne avec support des Promises natives et des requêtes paramétrées. Le pool gère la réutilisation des connexions.
Seed SQL versionné	Reproductibilité des environnements. N'importe quel développeur peut recréer la base en une commande (<code>npm run db:reset:demo</code>).